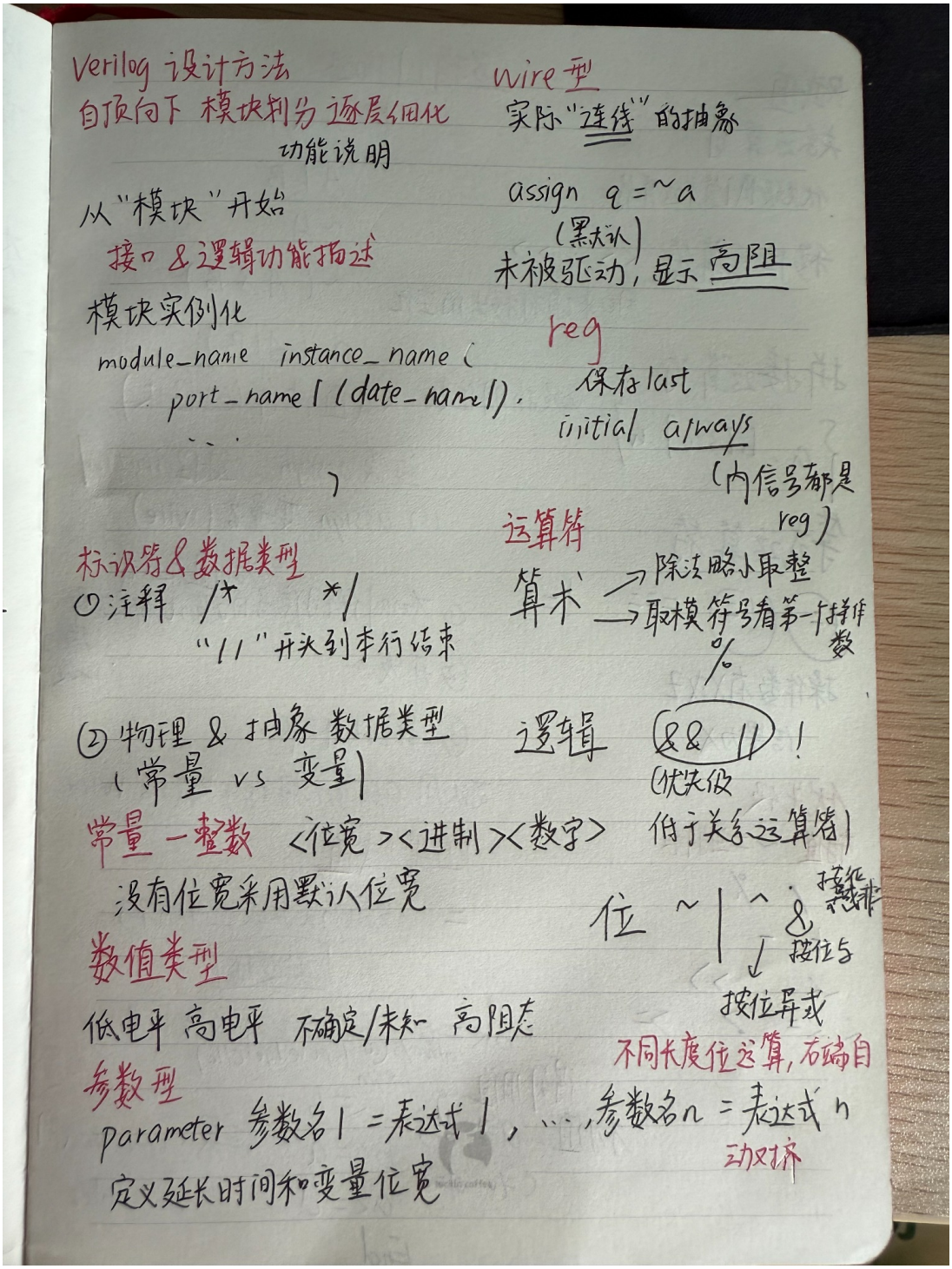
数字逻辑设计 · Verilog HDL

课件与三页手写讲义整合｜原图就近保留

原始笔记 ①｜模块、数据类型、运算符

这一页对应“自顶向下设计 → 模块实例化 → wire/reg → 数值与运算符”。



一、先把 Verilog 当作“描述硬件”

核心主线：需求 → 自顶向下划分模块 → 定义输入/输出与功能 → 编写可综合 RTL → 仿真验证 → 综合得到门级电路。代码语句是在描述电路，不是普通软件程序的一行行串行执行。

- 结构描述：模块/门的连接；数据流描述：assign 与表达式；行为描述：always/if/case。实际设计可混用。（课件第 5–10、14–18 页）
- module 是基本设计单元；实例化表示把子模块放进上层电路，命名端口 .port(signal) 是接线，不是函数传参。

```
module and2(input wire a, b, output wire y);
    assign y = a & b;
endmodule

module top(input wire a, b, output wire led);
    and2 u_and2 (.a(a), .b(b), .y(led));
endmodule
```

二、数据类型：先看“谁在驱动它”

- wire：线网，必须由连续赋值、模块输出等持续驱动；仿真中悬空线网通常为 z。
- reg：Verilog 过程赋值目标，写在 always/initial 的左侧；它是语言类型，不代表综合后一定是物理寄存器。
- parameter：编译时常量，常用于位宽、计数上限；常量建议明确位宽，如 8'hA2、4'b1010。

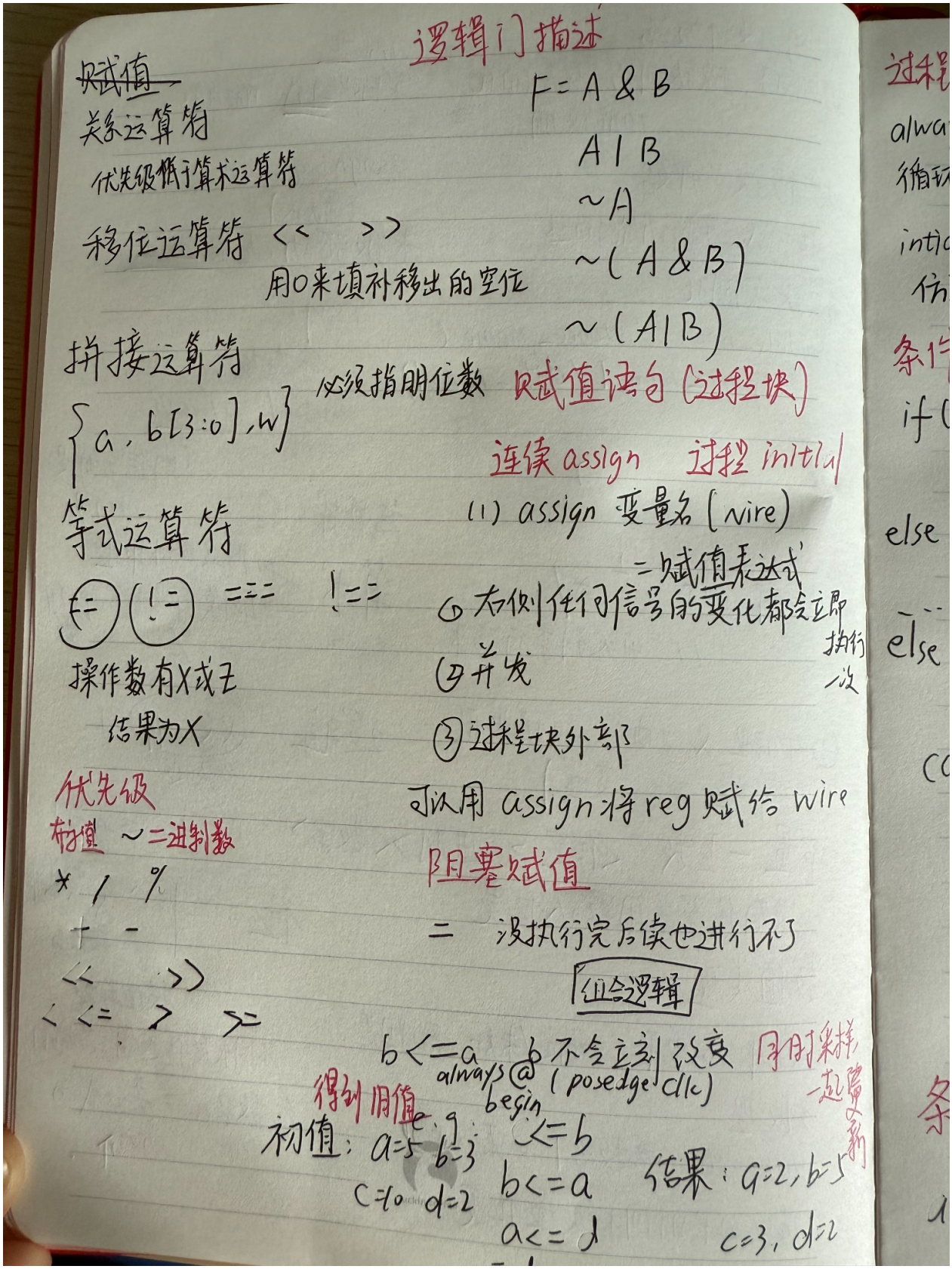
> 判断口诀：assign 左边一般为 wire；always 左边在经典 Verilog 中声明为 reg。组合逻辑 always @(*) 的 reg 输出也可能综合为纯组合电路。（课件第 21–30 页）

四态与声明

0、1、x（未知）、z（高阻）是仿真的四态。wire [7:0] bus; reg [3:0] q; 位宽写作 [高位:低位]。变量默认依赖隐式 wire 容易埋错，练习时尽量显式声明。

原始笔记 ②｜运算与两类赋值

这一页对应“位/逻辑运算 → assign → 阻塞与非阻塞赋值”。



三、运算符：位运算与逻辑运算要分开

- 位运算 &、|、~、^：逐位计算，保留向量位宽；逻辑运算 &&、||、!：先把操作数当真假，结果是 1 位。
- 移位 <<、>>：课件示例是逻辑移位；拼接 {a,b[3:0]} 须明确各部分位宽；整数除法截去小数。
- ==/!= 遇 x、z 可能得到 x；===/!== 做四态逐位比较，结果一定为 0 或 1，常用于仿真检查。

如 4'b1010 & 4'b1100 = 4'b1000；4'b1010 && 4'b1100 = 1'b1。复杂表达式用括号，别依赖背诵优先级。（课件第 32–37 页）

四、连续赋值、过程赋值与并行性

```
wire y;
assign y = (~a & b) | (a & ~b);  // XOR, continuous

reg q;
always @(posedge clk) begin
    q <= d;                      // sampled at clock edge
end
```

- assign 在过程块外，右侧变化会引起重新计算，多条 assign 并行存在；组合电路可用它表达。
- always 块之间并行；同一个过程块中的语句有顺序语义。always @(*) 用于组合逻辑，边沿触发 always 用于时序逻辑。

阻塞 = 与非阻塞 <= 的关键区别

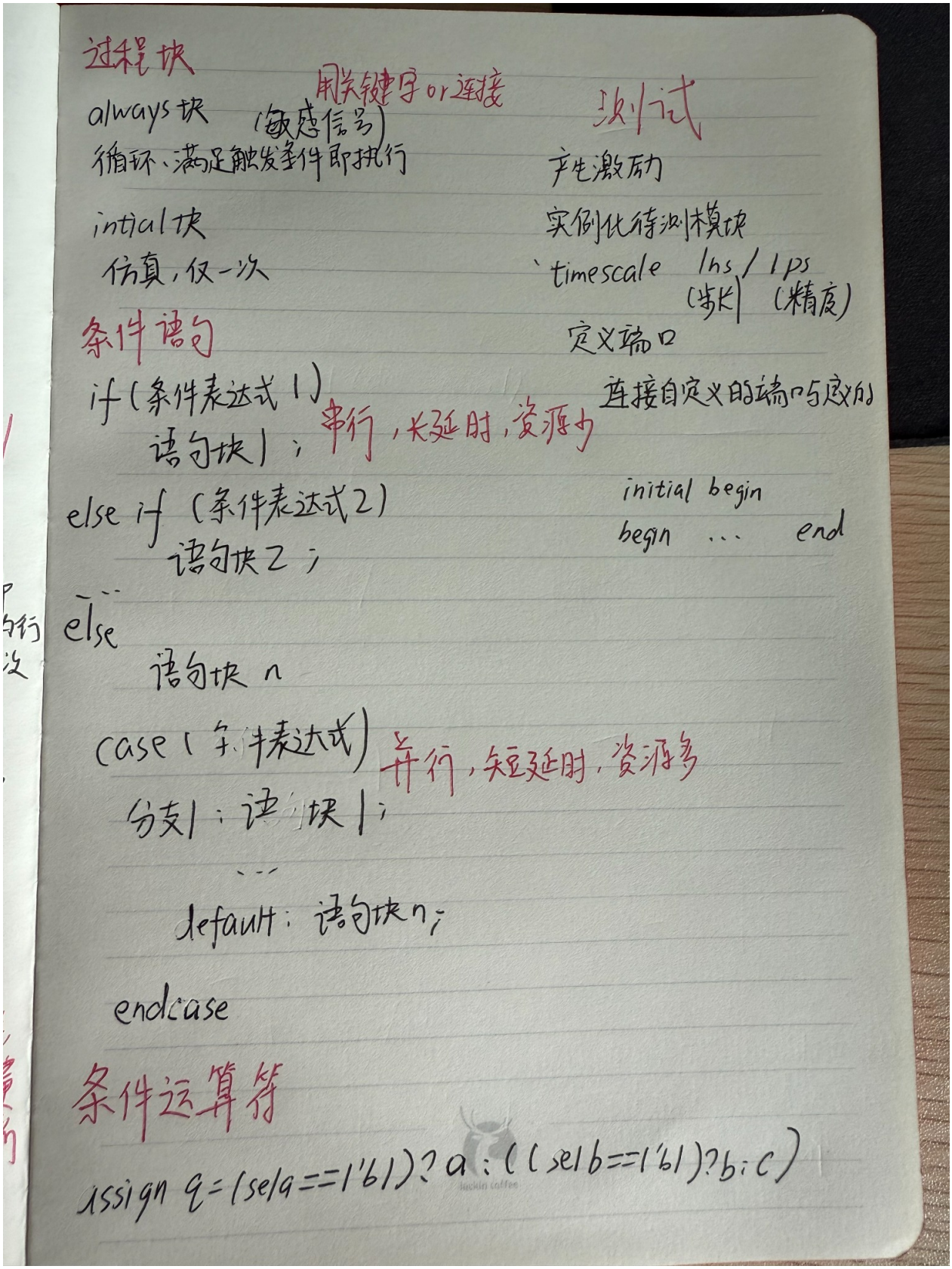
阻塞赋值 =：本语句执行时立即更新左值，后续语句可读到新值；组合逻辑过程块通常使用它。非阻塞赋值 <=：先计算右值，在当前时间步末统一更新；同一时钟沿中的后续语句读到旧值。时序逻辑通常使用它。（课件第 42–46 页）

```
// before edge: a=5, b=3, c=10, d=2
always @(posedge clk) begin
    c <= b;
    b <= a;
    a <= d;
end
// after edge:  a=2, b=5, c=3
```

同一个时钟沿上三条语句均取“旧值”。这里的 <= 是非阻塞赋值；在 if 条件中写 a <= b 才是“小于等于”比较。

原始笔记 ③｜过程块、选择与测试

这一页对应“always/initial → if/case → ?: → Testbench”。



五、过程块与组合逻辑写法

- always 会随敏感事件反复触发；组合逻辑用 @(*) 自动覆盖读到的输入，避免手写敏感列表遗漏。initial 通常作为仿真激励执行一次。
- 组合逻辑的每条执行路径都必须对输出赋值，否则旧值需被保留，通常会推断出锁存器。

```
module mux2(input wire sel, a, b, output reg y);
    always @(*) y = sel ? b : a;
endmodule
```

这里对所有输入情况都给 y 赋值，因此不会产生锁存器。也可以写 assign y = sel ? b : a，并把输出端口改为 wire。

六、if、case 与条件运算符

- if/else if 通常表示优先判断；case 按匹配项选择，建议写 default。多条语句用 begin/end；case 以 endcase 结束。
- ?: 是组合选择表达式，可嵌套，但层数多时可读性和时序可能变差。

```
always @(*) begin
    case (sel)
        2'b00: y = a;
        2'b01: y = b;
        2'b10: y = c;
        default: y = d;
    endcase
end
```

课件第 51 页的“if 串行、case 并行；一个省资源、一个延时短”只能作入门直觉，不能当作必然结论。综合结果取决于条件是否互斥、代码写法、目标器件和优化。

七、Testbench：用激励验证，而不是上板电路

Testbench 通常是无端口的顶层：实例化待测模块 DUT，用 initial 施加输入，在仿真中观察输出；激励信号可声明为 reg，DUT 输出连到 wire。`timescale 1ns/1ps 表示时间单位/精度。（课件第 55–58 页）

```
`timescale 1ns/1ps
module tb_mux2;
    reg sel, a, b;
    wire y;
    mux2 dut (.sel(sel), .a(a), .b(b), .y(y));
    initial begin
        a=1'b0; b=1'b1; sel=1'b0;
        #5 sel=1'b1;
        #5 $finish;
    end
endmodule
```

课件与手写笔记的三个纠偏点

- 课件第 48 页把 always 中赋值的 sum 声明为 output wire；经典 Verilog 中应改为 output reg，且其示例的模块端口末尾还需合法语法。
- “reg 都需要复位”不成立。是否需要复位取决于电路需求；组合逻辑输出本来不需要寄存器复位。
- 手写讲义的“initial 块用于仿真、只执行一次”适合入门常见用法；部分 FPGA 工具支持特定初始化，不能一概说 initial 永远不可综合。

资料依据：上传课件《Verilog 硬件描述语言》（59 页）与三张课堂手写讲义。补充与纠偏已尽量控制在本课范围内。